

DDP-Guided Residual Reinforcement Learning for Torque-Controlled Robot Trajectory Tracking under Model Mismatch

Anonymous Authors

Abstract—Trajectory optimization provides efficient and interpretable robot motions, but model mismatch and multirate deployment can prevent accurate reproduction on hardware. Increasing model fidelity is costly, whereas end-to-end torque learning discards useful task and feedback structure. This paper presents a differential dynamic programming (DDP)-guided residual reinforcement learning framework that retains the nominal DDP controller and learns only bounded residual torques. The stored DDP solution structures the policy inputs and learning objectives, while a dual-endpoint interface converts low-rate policy outputs into piecewise-linear torque commands for the high-rate loop. The method therefore compensates for mismatch between planning and execution without relearning the task from scratch. Simulation demonstrated faster learning and more accurate tracking than direct torque reinforcement learning. On a Franka Emika Panda, the proposed method achieved the lowest mean errors in end-effector position and orientation and the lowest root-mean-square (RMS) joint-position error across training and held-out trajectories, substantially outperforming both nominal and friction-compensated DDP. The dual-endpoint design also produced smoother torque-command evolution than zero-order hold. The hardware tests further demonstrated that this tracking advantage persisted under unmodeled payloads, including one beyond the training distribution. These results indicate that structured residual learning can provide a practical bridge from optimized motion to hardware execution without replacing the nominal controller.

I. INTRODUCTION

Model-based optimal control generates dynamically feasible robot motion by optimizing state and control trajectories subject to dynamics and physical constraints. The resulting sequences define a reference trajectory and feedforward command for a feedback controller that must reproduce the planned motion on hardware. At the joint torque level, optimization can exploit coupled robot dynamics and actuator limits to shape motion and interaction behavior [1], [2]. Such optimal control problems are commonly solved using direct transcription methods, including collocation [3], [4], or shooting-based methods.

Torque-level optimization, however, depends strongly on model fidelity. Position- and velocity-controlled interfaces typically use low-level motion servos to attenuate plant uncertainty. Direct joint torque control has no equivalent low-level motion servo, so modeling errors in inertia, bias forces, joint friction, payload properties, and actuator dynamics directly affect the commanded torque and closed-loop motion. Uncertainty in robot and payload parameters therefore remains a longstanding control challenge [5]. Consequently, discrepancies generally remain between the tractable nominal model used for optimization and the physical plant. Increasing

model fidelity raises the burden of system identification and optimization without necessarily eliminating residual mismatch.

Learning-based methods offer several ways to mitigate model mismatch. Direct torque reinforcement learning (Direct RL) maps observations to joint-torque commands [6] and can learn unmodeled effects, but it must also rediscover task structure and stabilizing behavior. Model-based learning retains stronger priors: guided policy search trains policies from locally optimized trajectories [7], while Lee et al. learn local dynamics corrections with Gaussian processes [8]. Residual policy learning instead adds a learned correction to an existing controller. Representative implementations augment hand-designed or model-predictive controllers [9], a Cartesian position controller [10], and a data-informed incremental tracking policy [11]. For policies trained in simulation, dynamics randomization can further support sim-to-real transfer [12]. Joint-torque residual learning, especially as a correction to optimizer-generated torques, remains comparatively underexplored. In this setting, conventional tracking rewards omit the optimizer's local value information and consequently weight tracking errors without reference to their effect on the nominal objective. Deployment also poses a multirate challenge: a low-rate policy must drive a high-rate torque loop without the step changes introduced by a zero-order hold [13]. These gaps motivate combining residual learning with a nominal optimizer that provides both an executable controller and structured learning signals.

Differential dynamic programming (DDP) is selected for this role because its second-order shooting method efficiently computes a nominal trajectory together with time-varying feedback gains and local value information [14]. The proposed framework retains the resulting DDP controller and learns only bounded residual torques. The stored trajectory and local value information structure the policy inputs and model-derived training signals. A dual-endpoint interface connects the low-rate policy outputs to the high-rate torque loop. Robustness to deployment mismatch is promoted by randomizing plant, payload, sensing, and timing conditions during training.

The main contributions are threefold:

- 1) A DDP-guided residual learning formulation retains the nominal controller's feedforward and feedback structure while allowing the planner to omit difficult-to-model effects such as joint friction. The stored DDP solution structures the policy inputs and auxiliary supervision, while local value information shapes the tracking reward.
- 2) A dual-endpoint residual interface connects a low-rate

neural policy to the high-rate torque loop through endpoint filtering and interpolation. An action-consistency loss aligns predictions across adjacent policy intervals.

3) A staged evaluation compares the method with Direct RL, nominal DDP, DDP with model-based friction compensation, joint velocity control, and a residual baseline using zero-order hold. The hardware evaluation also examines held-out trajectories and unmodeled payloads.

These evaluations demonstrate improved tracking accuracy and smoother torque-command evolution. Sections II–V present the problem, method, experiments, and conclusion, respectively.

II. PRELIMINARIES AND PROBLEM FORMULATION

A. Robot Dynamics and Model Mismatch

Let $\mathbf{q} \in \mathbb{R}^{n_q}$ denote the generalized coordinates, $\mathbf{v} = \dot{\mathbf{q}} \in \mathbb{R}^{n_q}$ the generalized velocities, and $\mathbf{x} = [\mathbf{q}^\top, \mathbf{v}^\top]^\top \in \mathbb{R}^{n_x}$ the state. The model-based trajectory optimizer uses a nominal model of rigid-body dynamics

$$\hat{\mathbf{M}}(\mathbf{q})\dot{\mathbf{v}} + \hat{\mathbf{h}}(\mathbf{q}, \mathbf{v}) = \mathbf{S}^\top \mathbf{u} + \hat{\mathbf{J}}_c(\mathbf{q})^\top \hat{\boldsymbol{\lambda}}, \quad (1)$$

where hats denote nominal-model quantities, $\hat{\mathbf{M}}$ is the inertia matrix, $\hat{\mathbf{h}}$ collects bias forces, $\mathbf{u}$ is the planned actuator command, and $\mathbf{S}^\top$ maps actuator commands to generalized forces. The product $\hat{\mathbf{J}}_c^\top \hat{\boldsymbol{\lambda}}$ represents optional constraint forces, including modeled contact forces. Joint friction is omitted from the nominal planner because its nonlinear behavior is difficult to model accurately over the robot's operating range. Accordingly, the execution dynamics include friction and other effects that are simplified or absent in (1):

$$\begin{aligned} \mathbf{M}(\mathbf{q})\dot{\mathbf{v}} + \mathbf{h}(\mathbf{q}, \mathbf{v}) + \boldsymbol{\tau}_f(\mathbf{v}) + \mathbf{D}\mathbf{v} + \boldsymbol{\tau}_{\text{unm}}(\mathcal{H}_t, t) \\ = \mathbf{S}^\top \boldsymbol{\tau}_{\text{cmd}} + \mathbf{J}_c(\mathbf{q})^\top \boldsymbol{\lambda}, \end{aligned} \quad (2)$$

where $\boldsymbol{\tau}_{\text{cmd}}$ is the joint-torque command applied during execution, and unhatted quantities describe the physical execution dynamics, including changes in inertia and bias terms caused by the payload. The terms $\boldsymbol{\tau}_f$ and $\mathbf{D}\mathbf{v}$ represent friction and viscous damping, $\mathcal{H}_t$ denotes recent state and command history, and $\boldsymbol{\tau}_{\text{unm}}$ collects history-dependent execution effects such as timing delays and errors in actuator torque tracking. This mismatch between the planner's nominal dynamics and the physical torque balance degrades hardware tracking, motivating learned residual compensation.

B. Nominal Optimal Control

Given a horizon of N control intervals and $N + 1$ state knots, the nominal trajectory is generated by solving the following constrained discrete-time optimal control problem (OCP):

$$\min_{\mathcal{X}, \mathcal{U}} J(\mathcal{X}, \mathcal{U}) = \ell_N(\mathbf{x}_N) + \sum_{k=0}^{N-1} \ell_k(\mathbf{x}_k, \mathbf{u}_k) \quad (3a)$$

$$\text{s.t. } \mathbf{x}_0 = \mathbf{x}_{\text{init}}, \quad (3b)$$

$$\mathbf{x}_{k+1} = \mathbf{f}_{\hat{\boldsymbol{\theta}}}(\mathbf{x}_k, \mathbf{u}_k), \quad (3c)$$

$$\mathbf{c}_k(\mathbf{x}_k, \mathbf{u}_k) \leq 0. \quad (3d)$$

Here, J is the objective; ℓ_k and ℓ_N are the stage and terminal costs; and $\mathcal{X} = \{\mathbf{x}_0, \dots, \mathbf{x}_N\}$ and $\mathcal{U} = \{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}\}$ denote the state and control sequences, with $\mathbf{x}_k \in \mathbb{R}^{n_x}$ and $\mathbf{u}_k \in \mathbb{R}^{n_u}$. The discrete transition $\mathbf{f}_{\hat{\boldsymbol{\theta}}}$ is obtained by integrating the nominal dynamics in (1) with identified or approximate parameters $\hat{\boldsymbol{\theta}}$. The vector-valued function $\mathbf{c}_k$ compactly represents the physical and task constraints imposed by the planner, including joint limits, torque limits, and collision-avoidance requirements. The learned residual policy is introduced only after the nominal OCP has been solved. The OCP solution defines the reference trajectory, while the DDP backward pass supplies the feedback gains and value-function derivatives described next.

C. DDP Solution and Stored Local Quantities

DDP solves (3) through alternating backward passes based on local quadratic models and forward rollouts with line search. Detailed derivations of the DDP backward pass and forward rollout can be found in [15]. Let $V_k(\mathbf{x})$ denote the local cost-to-go at knot k . With tracking error defined as reference minus state, the converged solution provides the reference state and command, the feedback gain $\mathbf{K}_k^*$, and the first- and second-order value derivatives

$$\begin{aligned} \mathbf{x}_k^{\text{ref}} = \mathbf{x}_k^*, \quad \mathbf{u}_k^{\text{ref}} = \mathbf{u}_k^*, \quad \mathbf{K}_k^{\text{ref}} = \mathbf{K}_k^*, \\ \mathbf{p}_k = V_{\mathbf{x},k}^*, \quad \mathbf{P}_k = V_{\mathbf{xx},k}^*. \end{aligned} \quad (4)$$

Here, $*$ identifies quantities at the converged DDP solution; $\mathbf{p}_k$ and $\mathbf{P}_k$ are the local value gradient and Hessian used later for reward weighting.

D. Planned Torque Tracking with Residual Control

The nominal DDP controller combines feedforward torque and local feedback. Time-indexed $\mathbf{x}_{\text{ref}}(t)$, $\mathbf{u}_{\text{ref}}(t)$, and $\mathbf{K}(t)$ denote interpolated values of the stored knot sequences in (4). At an arbitrary state $\mathbf{x}$, the controller is

$$\boldsymbol{\tau}_{\text{DDP}}(t; \mathbf{x}) = \mathbf{u}_{\text{ref}}(t) + \mathbf{K}(t)(\mathbf{x}_{\text{ref}}(t) - \mathbf{x}). \quad (5)$$

During execution, $\boldsymbol{\tau}_{\text{DDP}}(t)$ denotes the tracking torque $\boldsymbol{\tau}_{\text{DDP}}(t; \mathbf{x}(t))$; the planned feedforward torque $\mathbf{u}_{\text{ref}}(t)$ is independent of the execution state.

The physical system, including its low-level torque loop, does not exactly obey the nominal transition model $\mathbf{f}_{\hat{\boldsymbol{\theta}}}$. Let $\mathcal{T}$ and $\dot{\mathcal{T}}$ denote the admissible sets for torque and its rate, respectively, and $\Pi_{\mathcal{T}, \dot{\mathcal{T}}}$ the corresponding projection. The applied command is the projection of the summed nominal DDP and residual torques:

$$\boldsymbol{\tau}_{\text{cmd}}(t) = \Pi_{\mathcal{T}, \dot{\mathcal{T}}}(\boldsymbol{\tau}_{\text{DDP}}(t) + \boldsymbol{\tau}_{\text{RL}}(t)), \quad (6)$$

$$-\boldsymbol{\tau}_{\text{res}, \text{max}} \preceq \boldsymbol{\tau}_{\text{RL}}(t) \preceq \boldsymbol{\tau}_{\text{res}, \text{max}}. \quad (7)$$

The objective is to learn $\boldsymbol{\tau}_{\text{RL}}$ so that the physical system tracks the DDP reference more accurately while retaining the nominal DDP controller as the base controller.

The additive command structure naturally supports implementation at separate update rates. The DDP controller with feedforward and feedback runs in the high-rate torque loop, whereas the neural residual policy is evaluated at a lower rate

and supplies bounded compensation. This design removes the requirement that neural network inference, observation preprocessing, and operating-system scheduling satisfy the timing constraints of the high-rate torque loop.

III. METHOD

Building on the decomposition of commanded torque into nominal and residual components in (6), the proposed method retains the DDP controller in the high-rate torque loop and learns bounded residual torques to compensate for mismatch between planning and execution. A multirate interface interpolates the two endpoints predicted by each low-rate policy update, while the observations and learning objectives draw on the nominal trajectory and stored DDP information. Reinforcement learning is combined with supervision from inverse dynamics and a consistency constraint across adjacent intervals to improve tracking without replacing the nominal controller.

A. Dual-Endpoint Action Parameterization

Let j index policy updates and let t_j denote the corresponding physical time. At each update, the policy produces a raw dual-endpoint action in normalized coordinates, $\tilde{\mathbf{a}}_j$, whose two subvectors correspond to the start and end of the policy interval. It is projected onto $\mathcal{A} = [-1, 1]^{2n_u}$ before execution:

$$\mathbf{a}_j = \Pi_{\mathcal{A}}(\tilde{\mathbf{a}}_j) = [(\mathbf{a}_j^0)^\top, (\mathbf{a}_j^1)^\top]^\top \in \mathcal{A}, \quad (8)$$

where $\mathbf{a}_j^0, \mathbf{a}_j^1 \in [-1, 1]^{n_u}$ parameterize the residual torques at the interval start and end, respectively. Let $\mathbf{D}_{\text{res}} = \text{diag}(\boldsymbol{\tau}_{\text{res}, \text{max}}) \in \mathbb{R}^{n_u \times n_u}$ denote the diagonal matrix of residual torque scales. The residual endpoints are

$$\boldsymbol{\tau}_{\text{RL}, j}^i = \mathbf{D}_{\text{res}} \mathbf{a}_j^i, \quad i \in \{0, 1\}. \quad (9)$$

The bounded residual augments rather than replaces the nominal DDP controller, thereby preserving its feedforward and feedback structure and confining learning to compensation for planning-to-execution mismatch.

B. Cascaded Endpoint Filtering and Interpolation

Before interpolation, the predicted endpoints are filtered sequentially. Let $\bar{\boldsymbol{\tau}}_{\text{RL}, j-1}^1$ denote the residual torque accepted at the end of the preceding policy interval and let $\alpha_f \in (0, 1]$ denote the filter coefficient. The accepted endpoints are

$$\begin{aligned} \bar{\boldsymbol{\tau}}_{\text{RL}, j}^0 &= (1 - \alpha_f) \bar{\boldsymbol{\tau}}_{\text{RL}, j-1}^1 + \alpha_f \boldsymbol{\tau}_{\text{RL}, j}^0, \\ \bar{\boldsymbol{\tau}}_{\text{RL}, j}^1 &= (1 - \alpha_f) \bar{\boldsymbol{\tau}}_{\text{RL}, j}^0 + \alpha_f \boldsymbol{\tau}_{\text{RL}, j}^1. \end{aligned} \quad (10)$$

Thus, the filtered interval-start torque becomes the filter state for the interval-end torque, forming a cascade that preserves their temporal order.

With N_{sub} low-level substeps per policy interval, the accepted endpoints define the residual command

$$\begin{aligned} \boldsymbol{\tau}_{\text{RL}}(t_j + m\Delta t_{\text{low}}) &= (1 - \beta_m) \bar{\boldsymbol{\tau}}_{\text{RL}, j}^0 + \beta_m \bar{\boldsymbol{\tau}}_{\text{RL}, j}^1, \\ \beta_m &= \frac{m}{N_{\text{sub}}}, m = 0, \dots, N_{\text{sub}} - 1. \end{aligned} \quad (11)$$

Here Δt_{low} is the low-level control period and $\Delta t_\pi = N_{\text{sub}} \Delta t_{\text{low}}$ is the policy period. This construction replaces a residual command applied through a zero-order hold (ZOH) with a piecewise-linear command while leaving neural network inference outside the high-rate torque loop.

C. Observation Design

The policy observation is partitioned into blocks for the current state, history, and reference preview:

$$\mathbf{o}_j = [(\mathbf{o}_j^{\text{cur}})^\top, \text{vec}(\mathbf{H}_j)^\top, \text{vec}(\mathbf{F}_j)^\top]^\top, \quad (12)$$

where $\text{vec}(\cdot)$ stacks rows, and $\mathbf{H}_j$ and $\mathbf{F}_j$ are the history and reference-preview matrices defined below. This partition separates instantaneous feedback, past tracking mismatch and timing information, and a preview of the nominal plan.

At policy update j , let $\mathbf{x}_j^{\text{obs}}$ denote the latest state estimate supplied to the policy; it may be noisy, delayed, or reused when no new estimate is available. Define the tracking error, normalized DDP tracking torque, and normalized feedforward torque as

$$\begin{aligned} \mathbf{e}_j &= \mathbf{x}_{\text{ref}}(t_j) - \mathbf{x}_j^{\text{obs}}, \\ \tilde{\boldsymbol{\tau}}_{\text{DDP}, j} &= \mathbf{D}_\tau^{-1} \boldsymbol{\tau}_{\text{DDP}}(t_j), \\ \tilde{\mathbf{u}}_{\text{ref}, j} &= \mathbf{D}_\tau^{-1} \mathbf{u}_{\text{ref}}(t_j), \end{aligned} \quad (13)$$

where $\boldsymbol{\tau}_{\text{max}}$ collects the joint torque limits, with $\mathbf{D}_\tau = \text{diag}(\boldsymbol{\tau}_{\text{max}})$.

The current block is

$$\mathbf{o}_j^{\text{cur}} = [\mathbf{e}_j^\top, (\mathbf{x}_j^{\text{obs}})^\top, \tilde{\boldsymbol{\tau}}_{\text{DDP}, j}^\top, \mathbf{a}_{j-1}^\top]^\top, \quad (14)$$

where $\mathbf{a}_{j-1}$ is the previous dual-endpoint action.

For the rows in the history and preview blocks, let $\mathbf{x}_{j,i}^{\text{obs}}$ denote the stored state estimate and define $t_{j,i}^{\text{h}} = t_j - i\Delta t_\pi$ and $t_{j,i}^{\text{f}} = t_j + i\Delta t_\pi$. The corresponding error and normalized torque terms are

$$\begin{aligned} \mathbf{e}_{j,i}^{\text{h}} &= \mathbf{x}_{\text{ref}}(t_{j,i}^{\text{h}}) - \mathbf{x}_{j,i}^{\text{obs}}, & \tilde{\boldsymbol{\tau}}_{j,i}^{\text{h}} &= \mathbf{D}_\tau^{-1} \boldsymbol{\tau}_{\text{DDP}}(t_{j,i}^{\text{h}}), \\ \mathbf{e}_{j,i}^{\text{f}} &= \mathbf{x}_{\text{ref}}(t_{j,i}^{\text{f}}) - \mathbf{x}_{j,i}^{\text{obs}}, & \tilde{\mathbf{u}}_{j,i}^{\text{f}} &= \mathbf{D}_\tau^{-1} \mathbf{u}_{\text{ref}}(t_{j,i}^{\text{f}}). \end{aligned} \quad (15)$$

For each stored history sample, let $\Delta t_{j,i}^{\text{obs}} \geq 0$ denote the age of the state estimate in seconds and $\ell_{j,i}^{\text{stale}} \in \{0, 1\}$ indicate whether the preceding available state estimate was reused. Let $n_{j,i}^{\text{miss}}$ count consecutive missed policy updates and define $\bar{n}_{j,i}^{\text{miss}} = n_{j,i}^{\text{miss}} / (N_{\text{miss}}^{\text{max}} + 1)$, where $N_{\text{miss}}^{\text{max}}$ is the maximum permitted number of consecutive misses. The history block is formed from rows

$$\mathbf{o}_{j,i}^{\text{hist}} = [(\mathbf{e}_{j,i}^{\text{h}})^\top, (\mathbf{x}_{j,i}^{\text{obs}})^\top, \mathbf{a}_{j-i}^\top, (\tilde{\boldsymbol{\tau}}_{j,i}^{\text{h}})^\top, \Delta t_{j,i}^{\text{obs}}, \ell_{j,i}^{\text{stale}}, \bar{n}_{j,i}^{\text{miss}}]^\top, \quad (16)$$

where N_{h} is the history length, i indexes the rows, and $\mathbf{a}_{j-i}$ was issued i updates earlier. The history matrix is $\mathbf{H}_j = [(\mathbf{o}_{j,1}^{\text{hist}})^\top; \dots; (\mathbf{o}_{j,N_{\text{h}}}^{\text{hist}})^\top]$.

The preview block is formed from rows

$$\mathbf{o}_{j,i}^{\text{fut}} = [(\mathbf{e}_{j,i}^{\text{f}})^\top, (\tilde{\mathbf{u}}_{j,i}^{\text{f}})^\top]^\top, \quad (17)$$

where N_{f} is the lookahead length. The preview matrix is $\mathbf{F}_j = [(\mathbf{o}_{j,1}^{\text{fut}})^\top; \dots; (\mathbf{o}_{j,N_{\text{f}}}^{\text{fut}})^\top]$. This matrix previews the reference motion and feedforward torque over the next N_{f} policy samples.

D. Policy Network Architecture

The network follows the partition in (12): a two-layer multilayer perceptron (MLP) processes the current block, while separate long short-term memory (LSTM) encoders preserve the ordering within the history and preview windows:

$$\begin{aligned} \mathbf{z}_j^{\text{cur}} &= \phi_{\text{cur}}(\mathbf{o}_j^{\text{cur}}), \\ \mathbf{z}_j^{\text{hist}} &= \phi_{\text{hist}}(\mathbf{H}_j), \\ \mathbf{z}_j^{\text{fut}} &= \phi_{\text{fut}}(\mathbf{F}_j). \end{aligned} \quad (18)$$

The three embeddings are concatenated and mapped by separate fusion MLPs for the actor and critic:

$$\begin{aligned} \mathbf{z}_j &= [(\mathbf{z}_j^{\text{cur}})^\top, (\mathbf{z}_j^{\text{hist}})^\top, (\mathbf{z}_j^{\text{fut}})^\top]^\top, \\ \mathbf{h}_j^\pi &= g_\pi(\mathbf{z}_j), \\ \mathbf{h}_j^V &= g_V(\mathbf{z}_j). \end{aligned} \quad (19)$$

For actor parameters θ , two endpoint-specific, one-hidden-layer heads map $\mathbf{h}_j^\pi$ to the action means $\mu_\theta^0(\mathbf{o}_j)$ and $\mu_\theta^1(\mathbf{o}_j)$, which together form $\mu_\theta(\mathbf{o}_j)$, the mean of a Gaussian raw action with learned diagonal covariance. Sampled actions, or the mean during deterministic evaluation, are projected by (8). For critic parameters ψ , a head maps $\mathbf{h}_j^V$ to $V_\psi(\mathbf{o}_j)$.

E. Reward Shaping with DDP Value Information

The reward at policy update j is the sum of tracking and residual regularization terms:

$$r_j = r_j^{\text{track}} + r_j^{\text{effort}} + r_j^{\text{smooth}}. \quad (20)$$

Here r_j^{track} is the tracking reward, while r_j^{effort} and r_j^{smooth} penalize residual effort and temporal variation, respectively.

Let $T(\epsilon; \Sigma)$ be the average of $\exp[-(\epsilon/\sigma)^2]$ over $\sigma \in \Sigma$. The tracking reward is

$$\begin{aligned} r_j^{\text{track}} &= w_q r_{q,j} + w_v r_{v,j} + w_p r_{ee,p,j} + w_R r_{ee,R,j}, \\ r_{\alpha,j} &= T(\epsilon_{\alpha,j}; \Sigma_\alpha), \quad \alpha \in \{q, v\}, \\ r_{ee,p,j} &= T(\|\mathbf{e}_{p,j}\|; \Sigma_p), \quad r_{ee,R,j} = T(\|\mathbf{e}_{R,j}\|; \Sigma_R). \end{aligned} \quad (21)$$

Here $r_{q,j}$ and $r_{v,j}$ are the rewards for joint position and velocity, and $\mathbf{e}_{p,j}$ and $\mathbf{e}_{R,j}$ are the end-effector position and orientation errors. The sets Σ_q , Σ_v , Σ_p , and Σ_R contain the corresponding tolerances; w_q , w_v , w_p , and w_R weight the four tracking terms.

The task-space effect of a joint tracking error depends on both the affected joint and the robot configuration. The joint rewards therefore use root-mean-square (RMS) errors weighted by local DDP value information, emphasizing state errors according to their effect on the nominal cost-to-go rather than treating all joints and state directions uniformly. Writing $\mathbf{e}_j = [\mathbf{e}_{q,j}^\top, \mathbf{e}_{v,j}^\top]^\top$ and partitioning $\mathbf{W}_j$ into q - v blocks, define

$$\begin{aligned} C_j &= \mathbf{e}_{q,j}^\top \mathbf{W}_{qv,j} \mathbf{e}_{v,j}, \\ E_{\alpha,j} &= \mathbf{e}_{\alpha,j}^\top \mathbf{W}_{\alpha\alpha,j} \mathbf{e}_{\alpha,j} + C_j, \quad \alpha \in \{q, v\}, \\ \epsilon_{\alpha,j} &= \sqrt{[E_{\alpha,j}]_+ / n_q}. \end{aligned} \quad (22)$$

Here C_j is the position-velocity coupling term in the weighted state-error metric, $[z]_+ = \max(z, 0)$, and n_q is the number of joints. Since $\mathbf{W}_j$ is symmetric, $E_{q,j} + E_{v,j} = \mathbf{e}_j^\top \mathbf{W}_j \mathbf{e}_j$. During training, $r_{q,j}$ and $r_{v,j}$ are averaged over the low-level samples within the policy interval.

The matrix $\mathbf{W}_j$ is computed from local DDP value information. Let $\mathcal{P}_+(\cdot)$ symmetrize a matrix, project it onto the positive semidefinite cone, normalize positive eigenvalues to unit mean, and clip them to $[w_{\min}, w_{\max}]$. Let $\mathcal{D}(\cdot)$ normalize absolute values to unit mean and then apply the same clipping. When both stored value derivatives are available, the matrix derived from the value function is

$$\mathbf{G}_j = (1 - \eta_g) \mathcal{P}_+(\mathbf{P}_j) + \eta_g \text{diag}(\mathcal{D}(\mathbf{p}_j)), \quad (23)$$

where $\mathbf{P}_j$ and $\mathbf{p}_j$ are the time-aligned Hessian and gradient from (4), and $\eta_g \in [0, 1]$ controls their relative weighting. If only one derivative is available, $\mathbf{G}_j$ uses that derivative; if neither is available, it uses diagonal weights obtained from the column norms of $\mathbf{K}(t_j)$. The final metric for the state error is

$$\mathbf{W}_j = (1 - \eta_v) \mathbf{I} + \eta_v \mathbf{G}_j, \quad (24)$$

where $\eta_v \in [0, 1]$ controls the strength of value weighting.

The effort and smoothness terms penalize the residual action at the start of the interval and its temporal variation:

$$\begin{aligned} \delta \mathbf{a}_j &= \mathbf{a}_j^0 - \mathbf{a}_{j-1}^0, \quad \delta^2 \mathbf{a}_j = \mathbf{a}_j^0 - 2\mathbf{a}_{j-1}^0 + \mathbf{a}_{j-2}^0, \\ r_j^{\text{effort}} &= -c_a (\mathbf{a}_j^0)^\top \mathbf{W}_{\tau,j} \mathbf{a}_j^0, \end{aligned} \quad (25)$$

$$\begin{aligned} r_j^{\text{smooth}} &= -c_d(j) (\delta \mathbf{a}_j)^\top \mathbf{W}_{\tau,j} \delta \mathbf{a}_j \\ &\quad - c_{dd}(j) (\delta^2 \mathbf{a}_j)^\top \mathbf{W}_{\tau,j} \delta^2 \mathbf{a}_j. \end{aligned} \quad (26)$$

Here $\delta \mathbf{a}_j$ and $\delta^2 \mathbf{a}_j$ are the first- and second-order finite differences of the action at the interval start, and c_a , $c_d(j)$, and $c_{dd}(j)$ set the effort, first-order smoothness, and second-order smoothness penalties. The smoothness coefficients increase near the nominal trajectory, favoring small residual corrections when tracking errors are already low.

The diagonal matrix $\mathbf{W}_{\tau,j}$ weights these penalties by the local sensitivity of joint acceleration to joint torque. It is defined as

$$\begin{aligned} \mathbf{W}_{\tau,j} &= \text{diag}(\mathbf{w}_{\tau,j}), \\ \mathbf{w}_{\tau,j} &= \mathbf{1} + \eta_\tau (\hat{\mathbf{w}}_{\tau,j} - \mathbf{1}), \\ \hat{\mathbf{w}}_{\tau,j} &= \mathcal{C}_{[w_{\min}, w_{\max}]}(\text{diag}(\mathbf{M}_\xi(\mathbf{q}(t_j))^{-1})). \end{aligned} \quad (27)$$

Here $\mathcal{C}_{[w_{\min}, w_{\max}]}(\cdot)$ normalizes positive entries to unit mean and clips them to $[w_{\min}, w_{\max}]$, $\mathbf{1}$ is the vector of ones, $\hat{\mathbf{w}}_{\tau,j}$ is the resulting normalized sensitivity, and $\eta_\tau \in [0, 1]$ controls the strength of sensitivity weighting. Here ξ denotes the dynamics parameters sampled for the current training episode, and $\mathbf{M}_\xi$ is the corresponding simulated inertia matrix. This weight is computed only for the training reward and therefore requires no inertia-matrix estimation during hardware execution.

F. Auxiliary Target from Inverse Dynamics

At time s and state $\mathbf{x}_s = [\mathbf{q}_s^\top, \mathbf{v}_s^\top]^\top$, the inverse-dynamics (ID) auxiliary residual target is

$$\boldsymbol{\tau}_{\text{res,ID}}(s; \mathbf{x}_s) = \boldsymbol{\tau}_{\text{ID}}(\mathbf{q}_s, \mathbf{v}_s, \Pi_{\tilde{\mathcal{Q}}}(\ddot{\mathbf{q}}_{\text{ref}}(s))) - \boldsymbol{\tau}_{\text{DDP}}(s; \mathbf{x}_s), \quad (28)$$

where $\boldsymbol{\tau}_{\text{ID}}$ uses the randomized simulation model active in the episode, and $\tilde{\mathcal{Q}}$ is the acceleration set used for target construction. The torque target is projected onto the residual-torque box $\mathcal{T}_{\text{res}}$ defined by (7) and then normalized to action coordinates:

$$\mathbf{a}_{\text{ID}}(s; \mathbf{x}_s) = \mathbf{D}_{\text{res}}^{-1} \Pi_{\mathcal{T}_{\text{res}}}(\boldsymbol{\tau}_{\text{res,ID}}(s; \mathbf{x}_s)). \quad (29)$$

During training, this target supervises only the interval-start action, $\mathbf{a}_{\text{ID}}^0(j) = \mathbf{a}_{\text{ID}}(t_j; \mathbf{x}(t_j))$; the endpoint action corresponds to an unobserved future state and is therefore not assigned an inverse-dynamics target.

G. Training Losses

The policy is optimized with proximal policy optimization (PPO) [16]. Let $\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{o}_j) = [(\boldsymbol{\mu}_j^0)^\top, (\boldsymbol{\mu}_j^1)^\top]^\top$ denote the actor mean. Auxiliary supervision is applied to this mean, not to sampled actions. For a PPO minibatch $\mathcal{B}$, the ID loss for the action at the interval start is

$$\mathcal{L}_{\text{ID}} = \frac{\sum_{j \in \mathcal{B}} \sum_{r=1}^{n_u} \chi_{j,r}^{\text{ID}} (\boldsymbol{\mu}_{j,r}^0 - \mathbf{a}_{\text{ID},r}^0(j))^2}{\sum_{j \in \mathcal{B}} \sum_{r=1}^{n_u} \chi_{j,r}^{\text{ID}}}, \quad (30)$$

where r indexes actuated joints and $\chi_{j,r}^{\text{ID}} \in \{0, 1\}$ indicates target validity. Thus, supervision from inverse dynamics acts only on $\boldsymbol{\mu}_j^0$, the mean of the normalized residual action at the interval start.

To enforce temporal consistency across adjacent intervals, the action predicted for the end of interval j is matched to the interval-start action predicted from the next observation. Let $\chi_j^{\text{AC}} \in \{0, 1\}$ indicate that the transition remains in the same episode and has a valid successor observation. The action-consistency (AC) loss is

$$\mathcal{L}_{\text{AC}} = \frac{\sum_{j \in \mathcal{B}} \chi_j^{\text{AC}} \|\boldsymbol{\mu}_j^1 - \text{sg}(\boldsymbol{\mu}_{\boldsymbol{\theta}}^0(\mathbf{o}_{j+1}))\|_2^2}{n_u \sum_{j \in \mathcal{B}} \chi_j^{\text{AC}}}, \quad (31)$$

where $\text{sg}(\cdot)$ denotes stop-gradient. Thus, the prediction at the start of interval $j+1$ is treated as a fixed target during each update.

The total loss is

$$\mathcal{L} = \mathcal{L}_{\text{PPO}} + \lambda_{\text{ID}}(k_{\text{env}}) \mathcal{L}_{\text{ID}} + \lambda_{\text{AC}}(k_{\text{env}}) \mathcal{L}_{\text{AC}}, \quad (32)$$

where k_{env} is the cumulative number of environment transitions. For $z \in \{\text{ID}, \text{AC}\}$, λ_z decays linearly from its initial value to zero over the annealing horizon K_z and remains zero thereafter. The auxiliary losses affect only policy optimization; they do not alter the environment reward or deployed control law.

H. Domain Randomization for Sim-to-Real Transfer

At each episode start, the plant dynamics, payload, initial-state perturbation, and residual torque gain are sampled as specified in Table I; the plant, payload, and gain parameters then remain fixed. Payload properties are parameterized by mass, center of mass (COM), and an equivalent radius used to compute inertia. The DDP plan and nominal controller are not recomputed, so the residual policy must compensate for the sampled mismatch. Measurement noise, stale state estimates, and missed policy updates are sampled online; a missed update holds the last accepted residual endpoint for one policy interval.

IV. SIMULATIONS AND EXPERIMENTS

The evaluation progresses from simulation comparisons to hardware validation. The data used in this paper are available at <https://anonymous.4open.science/r/DDP-guided-residual-RL-D50B>.

A. Simulation Setup

The simulation used a MuJoCo [17] Panda model and 21 DDP plans for joint-space and Cartesian tasks, including reaching, orientation changes, and motions across a large workspace. Policies were trained with vectorized PPO; all simulation evaluations used this complete trajectory set.

The controller baselines were Nominal DDP and Direct RL. Direct RL issued a seven-dimensional joint-torque command at 100 Hz and held it constant between updates. It used the same trajectories, recurrent encoder, domain randomization, and reward as the proposed method but omitted DDP features; inverse-dynamics supervision was used only during training.

Training used 16 parallel environments. Residual policies were trained for 10^7 environment transitions, whereas Direct RL was continued to 10^8 for the extended-budget comparison. Each PPO rollout contained 1024 policy transitions per environment, with a 2048-sample minibatch, 10 epochs per update, $\gamma = 0.99$, and an initial learning rate of 10^{-4} . The low-level and policy periods were 1 and 10 ms; the history and preview windows each contained 10 policy samples. Table I specifies independently sampled randomization channels. The simulation metrics are mean step reward, RMS joint-position error, RMS joint-velocity error, RMS end-effector (EE) position error, RMS EE orientation error, RMS residual-torque magnitude, and RMS residual-torque rate.

B. Simulation Baseline Comparison and Ablation

1) *Residual RL versus Direct Torque RL:* Table II compares the methods under matched evaluation conditions. Nominal DDP used zero residual, and Direct RL used the best checkpoint from its completed 100-million-transition run. With one-tenth of that training budget, the proposed method achieved the lowest values in all three reported tracking-error metrics; its joint-position RMS error was 74.5% below Direct RL and 90.1% below Nominal DDP. Fig. 1 further shows that the mean return of the proposed method after 10 million transitions was 169.6% above that

TABLE I
TRAINING-TIME RANDOMIZATION DISTRIBUTIONS.

$U[a, b]$ and $\mathcal{N}(\mu, \sigma^2)$ denote uniform and Gaussian distributions.	
Quantity	Training distribution
Dynamics randomization	enabled with probability 0.8 per episode
Body mass and principal-inertia scales	$U[0.9, 1.1]$ per body and inertia component
Joint damping	$U[0, 0.05]$ N m s/rad per joint
Magnitude of joint dry friction	$U[0, 1.0]$ N m per joint
Payload randomization	enabled independently with probability 0.2 per episode
Payload mass	$U[0, 0.8]$ kg
Payload center of mass	$x, y \sim U[-0.03, 0.03]$ m; $z \sim U[0, 0.12]$ m
Equivalent payload radius	$U[0.02, 0.06]$ m
Initial-state perturbation ($\mathbf{q}, \mathbf{v}$)	$\mathcal{N}(0, (0.001)^2)$ rad, $\mathcal{N}(0, (0.005)^2)$ rad/s
Policy-observation noise ($\mathbf{q}, \mathbf{v}$)	$\mathcal{N}(0, (10^{-4})^2)$ rad, $\mathcal{N}(0, (5 \times 10^{-3})^2)$ rad/s
Feedback-state noise ($\mathbf{q}, \mathbf{v}$)	$\mathcal{N}(0, (2 \times 10^{-5})^2)$ rad, $\mathcal{N}(0, (10^{-3})^2)$ rad/s
State-estimate update	250 Hz; age jitter $U[0, 4]$ ms; stale probability 0.03 per policy update
Missed policy update	probability 0.04 per policy update; maximum hold of one policy interval (10 ms)
Residual torque gain	$U[0.95, 1.05]$ per joint and episode

of Direct RL, while its best return exceeded the maximum attained by Direct RL over 100 million transitions by 31.7%.

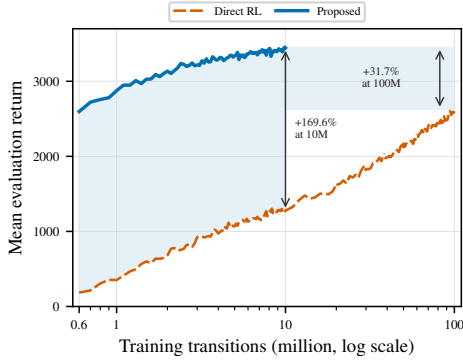


Fig. 1. Learning curves for the proposed method (10 million transitions) and Direct RL (100 million transitions).

2) *Staged Component Ablation*: Table III summarizes the cumulative ablation under matched plans, training budgets, rewards, and evaluation conditions. Fig. 2 shows that the proposed method improved more rapidly during early training than the MLP baseline and attained a maximum evaluation return comparable to the recurrent baseline with reward weighting and ID supervision. Table IV shows only modest differences between the proposed method and the recurrent configuration with reward weighting and ID supervision: the proposed method reduced all four tracking-error metrics and the residual-torque RMS, whereas the baseline achieved slightly higher reward and lower residual-torque-rate RMS. The advantage of the complete design was more evident in the hardware endpoint ablation.

C. Hardware Setup

Fig. 3 shows the hardware setup used for policy deployment without online adaptation. The torque-control loop ran

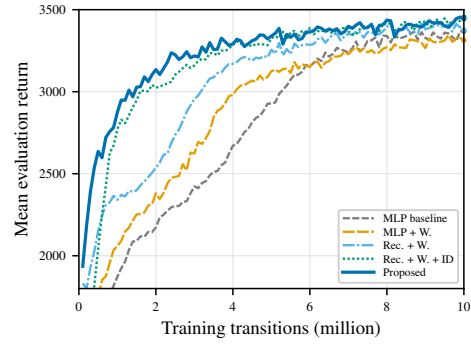


Fig. 2. Staged simulation ablation; the vertical axis starts at 1800 to resolve late-training differences.

at 1 kHz, while the residual policy was evaluated at 100 Hz ($N_{\text{sub}} = 10$) with a filter coefficient of $\alpha_f = 0.1$. The limits on residual and total torque were (5, 5, 5, 5, 2, 2, 2) and (87, 87, 87, 87, 12, 12, 12) N m, respectively; torque increments were limited to 1 N m per 1-ms control step.

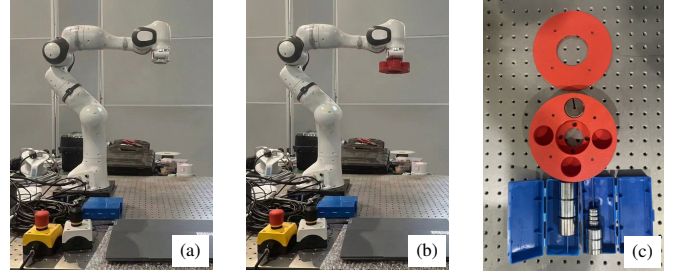


Fig. 3. Hardware setup: (a) unloaded robot; (b) robot with an attached payload; (c) configurable payload whose mass is adjusted by adding or removing cylindrical weights.

The hardware evaluation compared the proposed method with Nominal DDP, Friction DDP, and the built-in joint velocity controller. The three DDP-based methods used joint torque control, whereas the velocity controller served as a built-in servo baseline. Friction DDP follows a conventional model-based strategy [18]: it retains the nominal DDP state trajectory and feedback gains while adding feedforward friction compensation to the planned torque. The compensation is computed from an identified joint-friction model along the stored trajectory.

The evaluation covered the training set and five held-out trajectories excluded from training and model selection: a joint chirp, a horizontal Lissajous motion, an orientation task with a small translation, a diagonal reach toward a high frontal target, and a small tilted circle. Together, these tasks introduce frequencies, cross-axis coupling, reach directions, and tilted motion patterns not represented in the training set.

In addition to the simulation tracking metrics, the hardware evaluation reports the maximum end-effector position error. Let τ_J denote the measured joint torque; the commanded joint torque is τ_{cmd} , as defined in Section II-A. Measured-torque metrics comprise torque RMS and the RMS, 95th percentile, 99th percentile, and peak of the absolute torque rate. The endpoint ablation also reports the RMS rate of commanded torque and the RMS second derivative

TABLE II
SIMULATION CONTROLLER COMPARISON.

Method	Mean step reward	$\mathbf{q}$ RMS (deg)	EE position (mm)	EE orientation (deg)
Nominal DDP	2.42296 ± 0.00130	0.58668 ± 0.00011	0.88855 ± 0.00286	0.70421 ± 0.00029
Direct RL	2.71734 ± 0.00145	0.22675 ± 0.00007	1.58232 ± 0.00375	0.23551 ± 0.00244
Proposed	3.56400 ± 0.00104	0.05781 ± 0.00063	0.22595 ± 0.00116	0.07650 ± 0.00052

Mean $\pm$ sample standard deviation across three evaluations. Higher mean step reward is better; lower values are better for all other metrics.

TABLE III
STAGED SIMULATION ABLATION CONFIGURATION.

Method	W.	Rec.	ID	DEI
MLP baseline	–	–	–	–
MLP + W.	yes	–	–	–
Rec. + W.	yes	yes	–	–
Rec. + W. + ID	yes	yes	yes	–
Proposed	yes	yes	yes	yes

W.: reward shaped by DDP value information; Rec.: recurrent encoder; ID: inverse dynamics supervision at the interval start; DEI: dual-endpoint interface with cascaded filtering, interpolation, and AC supervision.

of measured torque.

D. Hardware Tracking and Torque Results

Table V summarizes the hardware tracking and torque metrics for the training and held-out sets.

1) *Tracking Accuracy and Generalization to Held-Out Trajectories*: On both trajectory sets, the proposed method achieved the lowest mean end-effector position and orientation errors and joint-position RMS error. Relative to Friction DDP, each of these errors decreased by at least 74.7% on the training set and 82.7% on the held-out set. The joint velocity controller achieved the lowest joint-velocity RMS error. These results indicate that the task-space pose and joint-position tracking advantage extends to trajectories excluded from training.

2) *Torque and Torque Rate Metrics*: Friction DDP minimized the aggregate torque-rate RMS and percentiles. The proposed method kept torque RMS within 1.6% of the minimum baseline value and achieved the lowest end-effector pose and joint-position errors, although it did not minimize the aggregate torque-rate metrics. Thus, the tracking gains did not require a substantial increase in torque magnitude.

E. Hardware Ablation of the Dual-Endpoint Interface

Table VI compares the complete dual-endpoint design with a ZOH residual under otherwise matched conditions.

Across both trajectory sets, the proposed method reduced every metric in Table VI by at least 12.9% relative to ZOH. The lower tracking errors and lower RMS values for commanded torque rate, measured torque rate, and the second derivative of measured torque indicate smoother torque-command evolution.

F. Robustness to Unmodeled Payloads

The payload evaluation compared the proposed method with Nominal DDP and Friction DDP at 0, 0.2, 0.6, and 1.0 kg. The nonzero payloads were assembled from the interchangeable components shown in Fig. 3(c). The 0.2- and 0.6-kg conditions lie within the payload-mass randomization

range in Table I; 1.0 kg exceeds the upper bound and is therefore out of distribution (OOD). For each nonzero payload, the nominal model continued to assume no external payload, introducing simultaneous errors in mass, center of mass, and inertia. Joint velocity control was excluded because its built-in servo compensates for the payload through a different mechanism. Fig. 4 summarizes the tracking trends.

The proposed method achieved the lowest mean end-effector position error, mean end-effector orientation error, and joint-position RMS error at every payload, including the OOD condition. Relative to both DDP baselines, these three tracking metrics were at least 71.7% lower. From 0 to 1.0 kg, its mean end-effector position error increased by 1.059 mm, compared with 3.823 and 3.962 mm for Nominal and Friction DDP, respectively. The tracking advantage therefore persisted beyond the training range despite simultaneous mass, center-of-mass, and inertia mismatch.

V. CONCLUSION

This work addressed planning-to-execution errors caused by mismatch between the nominal dynamics model and the physical robot, while retaining the DDP controller and learning only bounded residual torques. Stored DDP trajectory and value information are used to structure the policy inputs and learning objectives, while the dual-endpoint interface connects the low-rate policy to the high-rate torque loop through piecewise-linear interpolation.

In simulation, the method improved tracking over Nominal DDP and learned more efficiently than Direct RL. On hardware, it achieved the lowest end-effector pose and joint-position errors on the training and held-out sets and retained this advantage under a payload beyond the training randomization range. These tracking gains required no substantial increase in torque magnitude, although the method did not minimize every aggregate torque-rate metric. The dual-endpoint interface also produced smoother torque-command evolution than zero-order hold.

The framework assumes that DDP provides a valid nominal trajectory and local feedback structure; the learned residual compensates for model-induced execution mismatch around this nominal solution rather than task-level planning errors. Within this scope, the results indicate that DDP-guided residual learning can provide a practical means of transferring model-based optimal trajectories to torque-controlled hardware without replacing the nominal controller or evaluating the policy at the torque-loop rate.

REFERENCES

- [1] O. Khatib, "A unified approach for motion and force control of robot manipulators: The operational space formulation," *IEEE J. Robot. Autom.*, vol. 3, no. 1, pp. 43–53, Feb. 1987.

TABLE IV
STAGED SIMULATION COMPONENT ABLATION.

Method	Mean step reward	$\mathbf{q}$ RMS (deg)	$\dot{\mathbf{q}}$ RMS (deg/s)	EE pos. (mm)	EE ori. (deg)	τ_{RL} RMS (N m)	$\dot{\tau}_{RL}$ RMS (N m/s)
MLP baseline	3.348 $\pm$ 0.046	0.0576 $\pm$ 0.0019	0.867 $\pm$ 0.058	0.304 $\pm$ 0.023	0.0898 $\pm$ 0.0096	0.568 $\pm$ 0.038	23.38 $\pm$ 0.18
MLP + W.	3.356 $\pm$ 0.035	0.0584 $\pm$ 0.0013	0.918 $\pm$ 0.048	0.273 $\pm$ 0.026	0.0943 $\pm$ 0.0091	0.595 $\pm$ 0.038	52.05 $\pm$ 1.36
Rec. + W.	3.411 $\pm$ 0.022	0.0536 $\pm$ 0.0011	0.770 $\pm$ 0.044	0.237 $\pm$ 0.010	0.0916 $\pm$ 0.0059	0.597 $\pm$ 0.039	47.08 $\pm$ 0.12
Rec. + W. + ID	3.446 $\pm$ 0.027	0.0485 $\pm$ 0.0022	0.702 $\pm$ 0.056	0.262 $\pm$ 0.025	0.0897 $\pm$ 0.0110	0.584 $\pm$ 0.039	35.58 $\pm$ 0.19
Proposed	3.429 $\pm$ 0.007	0.0467 $\pm$ 0.0005	0.674 $\pm$ 0.009	0.210 $\pm$ 0.005	0.0756 $\pm$ 0.0020	0.578 $\pm$ 0.006	45.40 $\pm$ 0.19

Mean $\pm$ sample standard deviation over three evaluations. Higher mean step reward is better; lower values are better for all other metrics.

TABLE V
HARDWARE TRACKING AND TORQUE METRICS; LOWER IS BETTER.

Set	Method	Tracking error					Measured torque metrics				
		EE mean (mm)	EE max (mm)	Ori. mean (deg)	$\mathbf{q}$ RMS (deg)	$\dot{\mathbf{q}}$ RMS (deg/s)	$\dot{\tau}_J$ RMS (N m/s)	$P_{95}(\dot{\tau}_J)$ (N m/s)	$P_{99}(\dot{\tau}_J)$ (N m/s)	max $ \dot{\tau}_J $ (N m/s)	τ_J RMS (N m)
Training	Nominal DDP	1.811	4.017	1.014	1.379	2.419	35.993	75.303	136.739	574.524	4.204
	Friction DDP	1.307	2.653	0.774	1.007	2.207	34.916	73.991	131.293	504.671	4.195
	Joint velocity control	0.431	1.313	0.228	0.132	0.433	38.002	80.132	140.226	635.175	4.182
	Proposed	0.331	1.138	0.189	0.107	2.640	43.896	90.998	151.671	572.136	4.237
Held-out	Nominal DDP	2.565	5.452	1.348	1.216	2.422	28.450	59.606	106.483	431.478	4.009
	Friction DDP	1.962	3.706	1.184	0.968	2.302	28.102	58.793	104.360	438.124	3.976
	Joint velocity control	0.351	1.178	0.224	0.128	0.425	31.552	65.982	117.706	412.656	4.021
	Proposed	0.338	1.088	0.203	0.110	2.678	37.755	77.707	129.650	510.147	4.038

TABLE VI
HARDWARE ABLATION OF THE DUAL-ENDPOINT INTERFACE; LOWER IS BETTER.

Set	Method	EE mean (mm)	$\mathbf{q}$ RMS (deg)	$\dot{\mathbf{q}}$ RMS (deg/s)	τ_{cmd} RMS (N m/s)	$\dot{\tau}_J$ RMS (N m/s)	$\ddot{\tau}_J$ RMS (N m/s ²)
Training	ZOH residual	0.441	0.132	3.705	52.537	50.416	882.579
	Proposed	0.331	0.107	2.640	39.899	43.896	548.352
Held-out	ZOH residual	0.424	0.132	3.446	46.711	44.031	798.450
	Proposed	0.338	0.110	2.678	36.273	37.755	501.488

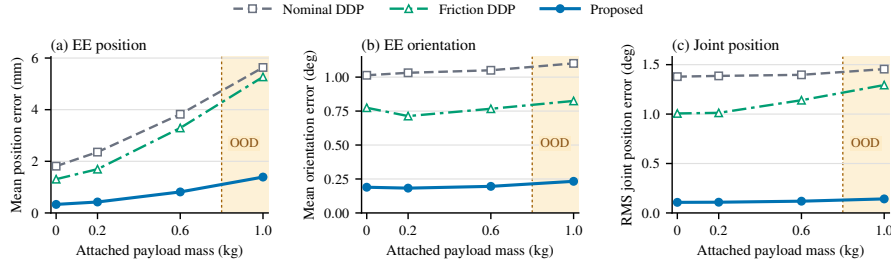


Fig. 4. Payload robustness; shading denotes the out-of-distribution (OOD) 1.0-kg condition.

- [2] A. Albu-Schäffer, C. Ott, and G. Hirzinger, “A unified passivity-based control framework for position, torque and impedance control of flexible joint robots,” *Int. J. Robot. Res.*, vol. 26, no. 1, pp. 23–39, Jan. 2007.
- [3] J. T. Betts, *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*, 2nd ed. Philadelphia, PA, USA: SIAM, 2010.
- [4] M. P. Kelly, “An introduction to trajectory optimization: How to do your own direct collocation,” *SIAM Rev.*, vol. 59, no. 4, pp. 849–904, Nov. 2017.
- [5] J.-J. E. Slotine and W. Li, “On the adaptive control of robot manipulators,” *Int. J. Robot. Res.*, vol. 6, no. 3, pp. 49–59, Sep. 1987.
- [6] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-end training of deep visuomotor policies,” *J. Mach. Learn. Res.*, vol. 17, no. 39, pp. 1–40, Apr. 2016.
- [7] S. Levine and P. Abbeel, “Learning neural network policies with guided policy search under unknown dynamics,” in *Adv. Neural Inf. Process. Syst.*, vol. 27, 2014.
- [8] G. Lee, S. S. Srinivasa, and M. T. Mason, “GP-ILQG: Data-driven robust optimal control for uncertain nonlinear dynamical systems,” arXiv:1705.05344, 2017.
- [9] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, “Residual policy learning,” arXiv:1812.06298, 2018.
- [10] T. Johannink et al., “Residual reinforcement learning for robot control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 6023–6029.
- [11] C. Li, F. Liu, Y. Wang, and M. Buss, “Data-informed residual reinforcement learning for high-dimensional robotic tracking control,” *IEEE/ASME Trans. Mechatron.*, vol. 30, no. 3, pp. 1681–1691, Jun. 2025.
- [12] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, “Sim-to-real transfer of robotic control with dynamics randomization,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 3803–3810.
- [13] R. Grandia et al., “Design and control of a bipedal robotic character,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2024.
- [14] C. Mastalli et al., “Crocodyl: An efficient and versatile framework for multi-contact optimal control,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2020, pp. 2536–2542.
- [15] Z. Huang, Y. Cheng, R. Liu, and M. Shan, “A hierarchical planning framework for self-assembly and self-reconfiguration of autonomous modular space robots,” *Robot. Auton. Syst.*, vol. 197, Art. no. 105273, 2026.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017.
- [17] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 5026–5033.
- [18] E. Madsen, O. S. Rosenlund, D. Brandt, and X. Zhang, “Adaptive feedforward control of a collaborative industrial robot manipulator using a novel extension of the generalized Maxwell-Slip friction model,” *Mech. Mach. Theory*, vol. 155, Art. no. 104109, Jan. 2021.